

Model Deployment Guide

This guide provides detailed instructions for deploying pre-trained machine learning models to the options pricing system.

Table of Contents

1. [Overview](#)
2. [Model Requirements](#)
3. [Local Development](#)
4. [Render Deployment](#)
5. [Model Loading Process](#)
6. [Troubleshooting](#)

Overview

The options pricing system uses pre-trained machine learning models that must be loaded from file paths. The models are:

- **XGBoost Max Depth 10** (Best model, MAE: 0.8093)
- **XGBoost Max Depth 5** (MAE: 1.6362)
- **5-Layer FFNN** (MAE: 4.6374)
- **3-Layer FFNN** (MAE: 8.8075)

Model Requirements

File Formats

- **XGBoost Models:** Native XGBoost format (`.model` extension)
- **Neural Networks:** Keras/TensorFlow HDF5 format (`.h5` or `.keras` extension)

Input Features (27 Total)

All models require exactly 27 features in this order:

1. `strike_price` - Option strike price
2. `implied_volatility` - Implied volatility
3. `zero_coupon_rate` - Risk-free interest rate
4. `index_dividend_yields` - Dividend yield
5. `option_type` - 1 for call, 0 for put
6. `time_to_maturity` - Time to expiration (years)
7. `underlying_asset_current_price` - Current stock price
- 8-27. `lag_1` through `lag_20` - Past 20 days' closing prices

Model Loading Code

XGBoost:

```
import xgboost as xgb
model = xgb.Booster()
model.load_model('path-to-your-model-file')
```

Neural Networks:

```
from tensorflow import keras
model = keras.models.load_model('path-to-your-model-file')
```

Local Development

Step 1: Create Models Directory

```
mkdir -p models
cd models
```

Step 2: Add Model Files

Place your trained model files in the `models/` directory:

```
models/
├── xgboost_depth_10.model
├── xgboost_depth_5.model
├── ffnn_5_layer.h5
└── ffnn_3_layer.h5
```

Step 3: Configure Environment Variables

Create or update `.env` file:

```
# Pre-trained Model Paths (Local Development)
XGBOOST_DEPTH_10_MODEL_PATH=./models/xgboost_depth_10.model
XGBOOST_DEPTH_5_MODEL_PATH=./models/xgboost_depth_5.model
FFNN_5_LAYER_MODEL_PATH=./models/ffnn_5_layer.h5
FFNN_3_LAYER_MODEL_PATH=./models/ffnn_3_layer.h5
```

Step 4: Test Model Loading

```
from src.models.xgboost_model import XGBoostMaxDepth10
from src.models.neural_network import FiveLayerFFNN

# Test XGBoost loading
xgb_model = XGBoostMaxDepth10()
print(f"XGBoost loaded: {xgb_model.is_trained}")

# Test Neural Network loading
nn_model = FiveLayerFFNN()
print(f"Neural Network loaded: {nn_model.is_trained}")
```

Render Deployment

Method 1: Persistent Disk (Recommended)

This is the best approach for production deployments.

Step 1: Create Persistent Disk

1. Go to your Render dashboard
2. Navigate to your service (Discord API or Trading Engine)
3. Click **"Disks"** → **"Add Disk"**
4. Configure:
 - **Name:** `model-storage`
 - **Size:** 1 GB (adjust based on your model sizes)
 - **Mount Path:** `/opt/render/project/src/models`

Step 2: Upload Model Files

Option A: Using Render Shell

1. Open Render Shell from your service dashboard
2. Navigate to the models directory:

```
bash
cd /opt/render/project/src/models
```

3. Upload files using `curl` or `wget` :

```
bash
# Example: Download from a URL
curl -o xgboost_depth_10.model https://your-storage-url/xgboost_depth_10.model
curl -o xgboost_depth_5.model https://your-storage-url/xgboost_depth_5.model
curl -o ffnn_5_layer.h5 https://your-storage-url/ffnn_5_layer.h5
curl -o ffnn_3_layer.h5 https://your-storage-url/ffnn_3_layer.h5
```

Option B: Using SCP (if SSH access is available)

```
scp models/xgboost_depth_10.model render-user@your-service:/opt/render/project/src/models/
scp models/xgboost_depth_5.model render-user@your-service:/opt/render/project/src/models/
scp models/ffnn_5_layer.h5 render-user@your-service:/opt/render/project/src/models/
scp models/ffnn_3_layer.h5 render-user@your-service:/opt/render/project/src/models/
```

Step 3: Configure Environment Variables

In Render dashboard, add environment variables:

```
XGBBOOST_DEPTH_10_MODEL_PATH=/opt/render/project/src/models/xgboost_depth_10.model
XGBBOOST_DEPTH_5_MODEL_PATH=/opt/render/project/src/models/xgboost_depth_5.model
FFNN_5_LAYER_MODEL_PATH=/opt/render/project/src/models/ffnn_5_layer.h5
FFNN_3_LAYER_MODEL_PATH=/opt/render/project/src/models/ffnn_3_layer.h5
```

Step 4: Verify Deployment

Check Render logs to confirm models are loaded:

```

Successfully loaded pre-trained model from: /opt/render/project/src/models/xg-
boost_depth_10.model
Successfully loaded pre-trained model from: /opt/render/project/src/models/xg-
boost_depth_5.model
Successfully loaded pre-trained model from: /opt/render/project/src/models/ffnn_5_laye
r.h5
Successfully loaded pre-trained model from: /opt/render/project/src/models/ffnn_3_laye
r.h5

```

Method 2: Google Cloud Storage

Use this method if you prefer cloud storage for model files.

Step 1: Upload to GCS

```

# Create bucket (if not exists)
gsutil mb gs://your-models-bucket

# Upload models
gsutil cp models/xgboost_depth_10.model gs://your-models-bucket/models/
gsutil cp models/xgboost_depth_5.model gs://your-models-bucket/models/
gsutil cp models/ffnn_5_layer.h5 gs://your-models-bucket/models/
gsutil cp models/ffnn_3_layer.h5 gs://your-models-bucket/models/

```

Step 2: Update Build Command

In `render-discord.yaml` and `render-trading.yaml`, update the build command:

```

buildCommand: |
  pip install -r requirements.txt
  mkdir -p /opt/render/project/src/models
  gsutil -m cp gs://your-models-bucket/models/* /opt/render/project/src/models/

```

Step 3: Configure Environment Variables

Same as Method 1, Step 3.

Method 3: Include in Repository (Not Recommended)

 **Warning:** Only use this method if your model files are small (<100 MB total) and don't contain proprietary data.

Step 1: Add Models to Repository

```

# Update .gitignore to allow model files
echo "!models/*.model" >> .gitignore
echo "!models/*.h5" >> .gitignore

# Add and commit
git add models/
git commit -m "Add pre-trained models"
git push

```

Step 2: Use Relative Paths

In `.env`:

```
XGBBOOST_DEPTH_10_MODEL_PATH=./models/xgboost_depth_10.model
XGBBOOST_DEPTH_5_MODEL_PATH=./models/xgboost_depth_5.model
FFNN_5_LAYER_MODEL_PATH=./models/ffnn_5_layer.h5
FFNN_3_LAYER_MODEL_PATH=./models/ffnn_3_layer.h5
```

Model Loading Process

Automatic Loading

Models are automatically loaded when the ensemble system initializes:

```
from src.ensemble import ModelEnsemble

# Initialize ensemble (loads all models)
ensemble = ModelEnsemble(use_automl=False)

# Check model load status
print(ensemble.model_load_status)
# Output: {
#   'XGBoost_Max_Depth_10': True,
#   'XGBoost_Max_Depth_5': True,
#   '5_Layer_FFNN': True,
#   '3_Layer_FFNN': True,
#   'Black_Scholes': True
# }
```

Manual Loading

You can also load models manually:

```
from src.models.xgboost_model import XGBoostMaxDepth10

# Load with explicit path
model = XGBoostMaxDepth10(model_path='./models/xgboost_depth_10.model')

# Or load after initialization
model = XGBoostMaxDepth10()
model.load('./models/xgboost_depth_10.model')
```

Error Handling

The system gracefully handles missing models:

```
# If a model fails to load, it will:
# 1. Print a warning message
# 2. Mark the model as not loaded in model_load_status
# 3. Return zero predictions for that model
# 4. Continue with other available models

# Example warning:
# "Warning: XGBoost Max Depth 10 model not loaded.
# Set XGBBOOST_DEPTH_10_MODEL_PATH environment variable."
```

Troubleshooting

Issue 1: Model File Not Found

Error:

```
FileNotFoundError: Model file not found: /path/to/model  
Please ensure the pre-trained model file is uploaded to this location.
```

Solutions:

1. Verify the file exists at the specified path
2. Check environment variable is set correctly
3. Ensure file permissions allow reading
4. For Render: Verify persistent disk is mounted correctly

Issue 2: Model Loading Failed

Error:

```
Exception: Failed to load model from /path/to/model: [error details]
```

Solutions:

1. **Wrong Format:** Ensure XGBoost models use `.model` extension, neural networks use `.h5`
2. **Corrupted File:** Re-download or re-upload the model file
3. **Version Mismatch:** Check library versions:

```
bash  
pip list | grep xgboost  
pip list | grep tensorflow
```
4. **Memory Issues:** Ensure sufficient RAM (models can be 50-100 MB each)

Issue 3: Missing Features

Error:

```
ValueError: Missing required features: ['lag_1', 'lag_2', ...]
```

Solutions:

1. Verify BigQuery connector is fetching lag features
2. Check `prepare_features()` method in `bigquery.py`
3. Ensure historical price data is available
4. Review feature engineering pipeline

Issue 4: Prediction Errors

Error:

```
Error predicting with XGBoost_Max_Depth_10: [error details]
```

Solutions:

1. Check input data format (must be pandas DataFrame)
2. Verify all 27 features are present

3. Ensure feature order matches model expectations
4. Check for NaN or infinite values in input data

Issue 5: Render Deployment Issues

Problem: Models not loading on Render

Solutions:

1. **Check Logs:** Review Render logs for detailed error messages
2. **Verify Paths:** Ensure absolute paths are used (`/opt/render/project/src/models/...`)
3. **Check Disk:** Verify persistent disk is mounted and has sufficient space
4. **Environment Variables:** Confirm all model path variables are set in Render dashboard
5. **Build Command:** Ensure build command downloads/copies models correctly

Issue 6: Performance Issues

Problem: Slow model loading or predictions

Solutions:

1. **Optimize Model Size:** Consider model compression or quantization
2. **Lazy Loading:** Load models only when needed
3. **Caching:** Cache predictions for frequently requested options
4. **Parallel Processing:** Use multiprocessing for batch predictions

Best Practices

1. **Version Control:**

- Tag model versions in filenames (e.g., `xgboost_depth_10_v1.2.model`)
- Document model training date and parameters

2. **Monitoring:**

- Log model load status on startup
- Track prediction latency
- Monitor model performance metrics

3. **Security:**

- Never commit model files to public repositories
- Use environment variables for all paths
- Encrypt sensitive model files
- Implement access controls on persistent disks

4. **Backup:**

- Keep backup copies of model files
- Store models in multiple locations (local + cloud)
- Document model provenance and training data

5. **Testing:**

- Test model loading in staging environment first
- Validate predictions against known test cases
- Monitor for prediction anomalies

Additional Resources

- **Research Paper:** [Pricing European Options with Google AutoML, TensorFlow, and XGBoost](https://arxiv.org/abs/2307.00476) (<https://arxiv.org/abs/2307.00476>)
- **Best Model:** [HuggingFace Repository](https://huggingface.co/juan-esteban-berger/XGBoost_European_Options_Pricing_MD_10) (https://huggingface.co/juan-esteban-berger/XGBoost_European_Options_Pricing_MD_10)
- **Render Documentation:** [Persistent Disks](https://render.com/docs/disks) (<https://render.com/docs/disks>)
- **XGBoost Documentation:** [Model Persistence](https://xgboost.readthedocs.io/en/stable/tutorials/saving_model.html) (https://xgboost.readthedocs.io/en/stable/tutorials/saving_model.html)
- **TensorFlow Documentation:** [Save and Load Models](https://www.tensorflow.org/tutorials/keras/save_and_load) (https://www.tensorflow.org/tutorials/keras/save_and_load)

Support

For additional help:

1. Check the main README.md for general setup instructions
2. Review the models/README.md for model-specific information
3. Open a GitHub issue with detailed error logs
4. Contact the repository maintainer

Last Updated: October 2025